```
Scenario: Should return scheduled flights in JSON form
  Given the following flights have been scheduled:
    | flightNumber | Departure | Destination | time  |
    | FH-101       | SYD       | MEL         | 06:15 |
    | FH-102       | MEL       | SYD         | 06:30 |
    | FH-223       | SYD       | LAX         | 06:00 |
    | FH-305       | MEL       | SFO         | 07:15 |
    | FH-234       | SYD       | LHR         | 09:25 |
    | FH-403       | SYD       | DBX         | 14:05 |
  When I request the International flights in JSON form
  Then I should receive the following flights:
  """
  [
    {"flightNumber":"FH-223",
     "departure":"SYD","destination":"LAX","time":"06:00"},
    {"flightNumber":"FH-305",
     "departure":"MEL","destination":"SFO","time":"07:15"},
    {"flightNumber":"FH-234",
     "departure":"SYD","destination":"LHR","time":"09:25"},
    {"flightNumber":"FH-403",
     "departure":"SYD","destination":"DBX","time":"14:05"}
  ]
  """
```

Set up the test data.

Call the web service.

Expected search results.

Providing sample output in JSON (or XML) form is a more technical approach that's not suitable for all scenarios. But when the audience is technical enough to understand the output, and the JSON format will provide added value over simply returning the information, it can be beneficial to express requirements in this format.

There are many other ways to implement a service architecture, both within the context of a single application and in a broader SOA strategy. In the Java world, EJB 3 and Spring Remoting are other frequently used approaches. Teams using a .NET stack will typically use WCF (Windows Communication Foundation) or WebAPI.

9.4 Defining and testing nonfunctional requirements

BDD isn't limited to purely functional requirements; the approach can also be used very successfully to discover and verify nonfunctional requirements. A *nonfunctional requirement* is traditionally defined as a (generally technical) requirement that's not related to a particular feature but relates to how the application operates as a whole. Typical nonfunctional requirements include performance, stability, accessibility, and security.

Just like functional requirements, nonfunctional requirements should deliver value in some identifiable way, typically by increasing revenue or reducing costs. An unresponsive website will cost an organization in customers and sales. According to Amazon, 100 ms of page latency results in a loss of 1% of sales.[3] Google found that a delay of half a second resulted in a traffic drop of 20%.[4]

---

[3] These figures were released in a presentation by Greg Linden, "Make Data Useful" (2006), https://sites.google.com/site/glinden/Home/StanfordDataMining.2006-11-28.ppt.

[4] Marissa Mayer, Web 2.0 Conference, 2006.